

OpenAI Codex CLI – Vyčerpávající příručka

Komplexní průvodce použitím, konfigurací, rozšířeními a osvědčenými postupy

Verze dokumentu: 1.0 **Datum:** 10. května 2026 **Pokrytá verze Codex CLI:** v0.130 (8. května 2026) **Cílová skupina:** Vývojáři, DevOps, technická publika **Pokrytí:** Všechny operační systémy (Windows, macOS, Linux) **Předpokládá se:** Codex CLI je již nainstalovaný a uživatel má funkční přihlášení (ChatGPT OAuth nebo `OPENAI_API_KEY`)

Obsah

1. [Úvod a kontext](#)
2. [Architektura a klíčové koncepty](#)
3. [Spouštění a globální CLI parametry](#)
4. [Subcommandy](#)
5. [codex exec – headless / CI režim](#)
6. [Slash příkazy v interaktivním TUI](#)
7. [Klávesové zkratky a Vim mode](#)
8. [Konfigurační hierarchie a config.toml](#)
9. [Profily \(\[profiles.*\] \)](#)
10. [Modely a model providers](#)
11. [Reasoning, verbosity, web search](#)
12. [AGENTS.md – paměť projektu](#)
13. [Approval policy a sandbox modes](#)
14. [Sandboxing detailně \(Linux / macOS / Windows / Docker\)](#)
15. [Environment policy a shell prostředí](#)
16. [Vestavěné nástroje \(Tools\)](#)
17. [Subagenti \(multi-agent, threads\)](#)
18. [Skills a \(deprecated\) custom prompts](#)
19. [Hooks](#)
20. [MCP integrace](#)
21. [Plugins a marketplaces](#)
22. [Notifikace](#)
23. [Memories \(perzistentní paměť\)](#)
24. [Auth a sessions](#)
25. [App-server, remote-control, IDE integrace](#)
26. [Logy, debug, tracing](#)
27. [Cost a usage tracking](#)
28. [CI/CD a codex-action](#)
29. [Best practices](#)
30. [Troubleshooting](#)
31. [Příloha A – Kompletní reference flagů](#)
32. [Příloha B – Reference klíčů config.toml](#)
33. [Příloha C – Reference environment proměnných](#)
34. [Příloha D – TOML šablony \(profil, provider, MCP server, agent\)](#)
35. [Příloha E – Glossář pojmů](#)

1. Úvod a kontext

OpenAI Codex CLI (open-source, repozitář `github.com/openai/codex`) je agentní nástroj v terminálu, který podobně jako Claude Code zvládá agentní programování: čte a edituje kód, spouští příkazy, prochází repozitář, koordinuje multi-step úkoly a integruje se s externími nástroji přes MCP. Liší se architektonicky (Rust core) i konfiguračně (TOML místo JSON, AGENTS.md místo CLAUDE.md, Apple Seatbelt / Linux Landlock místo zcela vlastního sandboxu).

1.1. Pro koho je tato příručka

Pro:

- vývojáře, kteří chtějí Codex pochopit z konfiguračního pohledu (config.toml, profiles, providers, MCP);
- DevOps / platform inženýry, kteří připravují Codex pro tým a CI/CD;
- techniky se zájmem o sandbox, bezpečnost a auditování agentního workflow.

Předpokládá se znalost terminálu (PowerShell, bash, zsh), Gitu a TOML / YAML konfigurací.

1.2. Co příručka pokrývá a co ne

Pokrývá:

- celé CLI: globální flagy, subcommandy, `codex exec`, slash příkazy, klávesové zkratky;
- konfiguraci v `~/ .codex/config.toml`, profiles, model providers;
- AGENTS.md, skills, plugins, hooks, multi-agent;
- MCP integraci (stdio + streamable HTTP);
- approval policy a sandboxing (Apple Seatbelt, Linux Landlock + seccomp, Windows hardening, Docker);
- headless režim a CI/CD (GitHub Actions, codex-action, codex-universal);
- IDE integraci a app-server protokol;
- logy, debug, troubleshooting, best practices.

Nepokrývá:

- instalaci a první spuštění (`brew install codex`, `npm i -g @openai/codex` atd.);
- internals modelů GPT-5/5.1/Codex (architektura, fine-tuning);
- detailní reverse engineering protokolu app-serveru.

1.3. Konvence v textu

- `codex` – jméno binárky.
- `~/ .codex/` – domovská složka. Na Windows obvykle `%USERPROFILE%\ .codex\`. Lze přepsat env proměnnou `CODEX_HOME`.
- `config.toml` – základní konfigurační soubor (`$CODEX_HOME/config.toml`).
-  – upozornění na bezpečnost.
-  – nenápadná past.
-  – praktický tip.

1.4. Codex vs Claude Code – mentální mapa pro orientaci

Pokud znáte Claude Code, pomůže vám tato hrubá tabulka analogií:

Koncept	Claude Code	Codex CLI
Konfigurace	settings.json (JSON)	config.toml (TOML)
Paměť projektu	CLAUDE.md	AGENTS.md
Headless mode	claude -p	codex exec
Pokračování relace	claude -c, claude --resume	codex resume, codex resume --last
Globální správa	claude config, claude mcp	flagy + codex mcp, codex login, codex plugin
Subagenti	.claude/agents/*.md	.codex/agents/*.toml
Skills	.claude/skills/<name>/SKILL.md	.codex/skills/<name>/SKILL.md
Slash příkaz	/clear, /init, /mcp	/clear, /init, /mcp
MCP	stdio + http + sse	stdio + streamable HTTP (rmcp)
Sandbox	vlastní permission engine	Seatbelt / Landlock+seccomp / Windows

Konkrétní detaily jsou samozřejmě jiné – podrobně v dalších kapitolách.

2. Architektura a klíčové koncepty

Codex CLI je napsaný v Rustu jako monolitický agent s několika subcommandy. Jádro řeší komunikaci s modelem (OpenAI Responses API nebo OpenAI Chat-compatible API), interpretaci tool calls (shell, edits, MCP), sandboxing a TUI. Doplnkové binární režimy (`codex app-server` , `codex mcp-server` , `codex remote-control`) zpřístupňují stejné jádro v JSON-RPC formátu, aby na něj mohla navazovat IDE rozšíření a desktopový klient.

2.1. Co je relace (session)

Když spustíte `codex` v adresáři, vznikne **interaktivní relace** s přiřazeným UUID. Každá relace má:

- **cwd** (a volitelně `--cd`),
- **přihlášený účet** (ChatGPT OAuth nebo API key),
- **aktivní model_provider + model** (možná z profilu),
- **approval policy + sandbox mode**,
- **načtený AGENTS.md** (z global + project lookup),
- **registrované MCP servery**,
- **transcript** v `~/ .codex/sessions/YYYY/MM/DD/rollout-*.jsonl` .

Relaci lze později obnovit: `codex resume --last` , `codex resume <SESSION_ID>` , případně forknout přes `codex fork` .

2.2. Hierarchie konfigurace

Codex čte konfiguraci v tomto pořadí:

1. **CLI flagy** (např. `--model`, `--ask-for-approval`, `-c key=val`) – nejvyšší priorita per-invocation.
2. **Aktivovaný profil** (`--profile <name>` → `[profiles.<name>]`).
3. **Project-level konfigurace** (`.codex/config.toml`) – aplikuje se pouze pro **trusted** projekt (`projects."<path>".trust_level = "trusted"`).
4. **Global config** (`$CODEX_HOME/config.toml`).
5. **Vestavěné defaulty**.

⚠ Project-level konfigurace u nedůvěryhodného projektu se ignoruje. Trust se nastavuje v `[projects."<path>"]`.

2.3. Tři vrstvy rozšiřitelnosti

Codex lze rozšiřovat:

- **Slash commandy** – ad-hoc makra (`/clear`, `/diff`, `/review`).
- **Custom prompts (deprecated)** – `~/.codex/prompts/*.md` s YAML frontmatterem, vyvolávané přes `/prompts:<name>`.
- **Skills** – nástupce custom prompts: adresáře v `~/.codex/skills/<name>/` s `SKILL.md`. Codex je vybírá automaticky podle popisu nebo explicitně přes `$skill-name`.
- **Subagenti (multi-agent)** – `.codex/agents/<name>.toml`, paralelní vlákna pro izolované úkoly.
- **Hooks** – deterministické skripty na lifecycle eventy.
- **MCP servery** – externí tooly (stdio/HTTP) přes Model Context Protocol.
- **Plugins** – distribuovatelný balíček obsahující skills, hooks, MCP servery, apps.

2.4. „Pravidlo nejmenšího překvapení“

- *Konfigurace v TOML, citlivá data NE.* `auth.json`, `OPENAI_API_KEY`, secrets vždy mimo git.
- *Sandboxujte rané fáze.* `read-only` + `untrusted` = bezpečné prozkoumání cizího kódu.
- *AGENTS.md je krátké memo, ne kniha.* Defaultně se ořezává na `project_doc_max_bytes`.
- *Profily jsou váš nejlepší kamarád.* Jeden pro „review“, druhý pro „CI“, třetí pro „lokální Ollama“.

3. Spouštění a globální CLI parametry

Základní volání:

```
codex                                # interaktivní TUI v aktuálním adresáři
codex "Vysvětli src/auth.cs"        # interaktivní s úvodním promptem
codex exec "Zhrň README"            # headless – odpověď a konec
```

3.1. Hlavní globální flagy

Flag	Význam
------	--------

<code>-m, --model <MODEL></code>	Vynutí konkrétní model (např. <code>gpt-5-codex</code> , <code>gpt-5.1</code> , <code>o4-mini</code>).
<code>-p, --profile <NAME></code>	Aktivuje profil z <code>[profiles.<NAME>]</code> .
<code>-c, --config <KEY=VAL></code>	Inline override hodnoty z <code>config.toml</code> . Lze opakovat. JSON parsování (číslo / bool / string).
<code>--cd <DIR></code>	Pracovní adresář před zpracováním.
<code>-i, --image <PATH></code>	Přiloží obrázek (PNG/JPEG/WebP). Lze opakovat / čárkový seznam.
<code>--oss</code>	Použije lokální OSS provider (typicky Ollama). Ekvivalent <code>-c model_provider="oss"</code> .
<code>--full-auto</code>	Plná automatizace – auto-approval pro vše, <code>sandbox workspace-write</code> . Ideální pro CI.
<code>--ask-for-approval <MODE></code>	<code>untrusted</code> / <code>on-request</code> / <code>never</code> (nebo granular podle config).
<code>--sandbox <MODE></code>	<code>read-only</code> / <code>workspace-write</code> / <code>danger-full-access</code> .
<code>--dangerously-bypass-approvals-and-sandbox</code>	 Vypne všechny brzdy. Pouze v Dockeru / VM.
<code>--search</code>	Zapne <code>web_search = "live"</code> pro aktuální běh.
<code>--enable <FEATURE></code>	Aktivuje feature flag (opakovatelný).
<code>--login</code>	Vyžádá login flow před prací.
<code>--remote <ws://...></code>	Připojí TUI k remote app-serveru přes WebSocket.

3.2. Použití `-C` pro override

Hodnoty `-C` mají JSON sémantiku, takže:

```
codex -c model="gpt-5-pro"
codex -c sandbox_mode='"read-only"'
codex -c 'sandbox_workspace_write.network_access=true'
codex -c approval_policy='"never"'
codex -c 'features.fast_mode=true'
```

 Pozor na escapování v různých shellech. V PowerShellu používejte single quotes nebo `--%` pro literál:

```
codex -c 'sandbox_workspace_write.network_access=true'
```

3.3. Profil + override v jednom volání

```
codex --profile review -c model_reasoning_effort='"high"' -c  
web_search='"live"'
```

3.4. Spuštění s OSS modelem (Ollama)

```
codex --oss -m "qwen3-coder:30b"
```

(Předpokládá `model_providers.oss` se správným `base_url`.)

3.5. Headless one-shot s API klíčem

```
OPENAI_API_KEY="sk-..." codex exec --full-auto \  
"Najdi a oprav null reference exception v Services/Order"
```

3.6. Spuštění s WebSocket remote app-serverem

```
codex remote-control --listen ws://0.0.0.0:7777  
# v jiném terminálu / CI:  
codex --remote ws://devbox:7777
```

🛡️ Provoz bez auth funguje jen na loopbacku. Pro non-loopback přidejte HMAC-signed JWT/JWS bearer auth (viz kapitola 25).

4. Subcommandy

Subcommand	Účel
codex	Interaktivní TUI.
codex exec (alias codex e)	Neinteraktivní spuštění (CI/CD, skripty).
codex exec review	Headless code review (presety: uncommitted, base-branch, commit, custom).
codex resume [SESSION_ID]	Pokračování interaktivní relace (--last, --all).
codex fork	Forkne minulou relaci do nového vlákna.
codex apply (alias codex a)	Aplikuje patch z Codex Cloud úlohy do pracovního stromu.
codex login	Login (ChatGPT OAuth, API key, device-code).
codex logout	Smaže lokální credentials.

codex mcp	Správa MCP serverů.
codex sandbox	Spuštění příkazu v sandboxu (subaliasy seatbelt, landlock).
codex plugin	Správa pluginů a marketplaces.
codex completion <shell>	Generuje shell completion (bash/zsh/fish/powershell).
codex update	Self-update binárky (od v0.128).
codex app-server	Spustí app-server (JSON-RPC stdio / WebSocket).
codex mcp-server	Codex jako MCP server (stdio).
codex remote-control	Headless remote-control endpoint (od v0.130).
codex debug	Diagnostika app-serveru (single V2 message).
codex execpolicy	Validace execpolicy rule files.
codex generate-ts	Generuje TypeScript protocol definitions.
codex --help / codex help	Plná nápověda.

4.1. codex login

```
codex login                                # interaktivní výběr metody
codex login --with-api-key < secret.txt # API key ze stdin
codex login --device-auth                 # device-code flow pro headless /
SSH / Docker
```

Credentials se ukládají do `$CODEX_HOME/auth.json`. Proměnná `OPENAI_API_KEY` má v některých scénářích přednost (zvláště v CI). Volba `preferred_auth_method = "apikey"` v `config.toml` přepíná default.

4.2. codex logout

```
codex logout
```

Smaže `auth.json` a invaliduje OAuth refresh token.

4.3. codex resume

```
codex resume                # interaktivní picker
codex resume --last         # nejnovější ze cwd
```

```
codex resume --all           # napříč libovolným adresářem
codex resume <SESSION_ID>    # konkrétní UUID
codex resume --last "Pokračuj refaktorem AuthService"
```

Manuálně lze načíst session přes `codex -c experimental_resume="<path k rollout-*.jsonl>"`.

4.4. codex fork

Vyvolá picker minulé relace a vytvoří nový thread odštěpený od libovolného turnu. Užitečné pro průzkum „co kdyby“ bez ztráty originálu.

4.5. codex apply

Aplikuje patch (typicky `unified diff` / git patch) z Codex Cloud úlohy. Oficiální workflow: úloha v Codex Cloud → výstup `patch` → `codex apply` lokálně.

4.6. codex mcp

Detail v kapitole 20.

```
codex mcp list
codex mcp add <name> -- <stdio_command> [args...]
codex mcp add <name> --url https://server/mcp [--bearer-token TOKEN]
codex mcp login <name>           # OAuth pro streamable HTTP
codex mcp remove <name>
```

4.7. codex sandbox

```
codex sandbox seatbelt -- ls -la           # macOS
codex sandbox landlock -- cargo build      # Linux
codex sandbox -- /bin/bash -c 'echo hello' # výchozí dle OS
```

Vrátí exit kód podřízeného příkazu, pokud sandbox neselže (pak vrací non-zero z důvodu blocku).

4.8. codex plugin

```
codex plugin marketplace add owner/repo
codex plugin marketplace add owner/repo --ref main
codex plugin marketplace add https://github.com/example/plugins.git --sparse
agents/plugins
codex plugin marketplace add ./local-marketplace-root
```



```
codex plugin marketplace upgrade
codex plugin marketplace remove <marketplace-name>
codex plugin install <plugin-name>
codex plugin list
codex plugin remove <plugin-name>
```

4.9. codex completion

```
codex completion bash > /etc/bash_completion.d/codex
codex completion zsh >> ~/.zsh_completions/_codex
codex completion fish > ~/.config/fish/completions/codex.fish
codex completion powershell | Out-String | Invoke-Expression
```

4.10. codex update

```
codex update          # self-update poslední stabilní
```

Update lze blokovat env var (firma kontroluje verze přes balíčkový manažer): nastavte

```
CODEX_DISABLE_AUTOUPDATE=1 (neověřeno; ověřte v codex update --help).
```

5. codex exec – headless / CI režim

Headless mód je primární vstupní bod pro automatizaci.

5.1. Základní pattern

```
codex exec "Najdi v projektu TODO komentáře a vrať je v markdown tabulce" \
--full-auto \
--json \
--output-last-message ./out.md
```

5.2. Klíčové flagy

Flag	Význam
--json	Výstup je JSONL – jeden event objekt na řádek.
-o, --output-last-message <FILE>	Zapíše finální zprávu agenta do souboru (kombinace s --json produkuje pouze JSON).
--output-schema <FILE>	JSON Schema, kterým se validuje finální výstup.

--include-plan-tool	Zapne plan tool během exec (užitečné pro multi-step úlohy).
--skip-git-repo-check	Povolí běh mimo git repo.
--full-auto	Auto-approval + workspace-write sandbox.
--ask-for-approval	Stejná sémantika jako globálně.
--sandbox	Stejná sémantika jako globálně.
--cd <DIR>	Pracovní adresář.
--profile <NAME>	Profil.

5.3. Příklad: code review v CI

```
codex exec review base-branch main \
  --json \
  --output-last-message review.md \
  --profile review
```

Presety pro `codex exec review`:

Preset	Účel
uncommitted	Review jen neverzovaných změn.
base-branch <ref>	Review proti uvedené větvi.
commit <sha>	Review jednoho commitu.
custom <instruction>	Volný prompt.

5.4. Strukturovaný výstup s `--output-schema`

```
cat > schema.json <<'JSON'
{
  "type": "object",
  "properties": {
    "issues": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "file": { "type": "string" },
          "line": { "type": "integer" },
```

```

        "severity": { "enum": ["info", "warn", "error"] },
        "message": { "type": "string" }
    },
    "required": ["file", "line", "severity", "message"]
}
}
},
"required": ["issues"]
}
JSON

```

```

codex exec "Najdi bezpečnostní problémy a vrať je dle schématu." \
  --output-schema schema.json \
  --output-last-message issues.json \
  --full-auto

```

Codex potom validuje finální zprávu proti schématu a v případě nesouladu vrací chybu.

5.5. Stream JSON událostí

S `--json` Codex emituje JSONL události:

Event	Význam
thread.started	Začala nová relace.
turn.started	Začal turn.
turn.completed	Konec turnu (v usage: input_tokens, cached_input_tokens, output_tokens).
turn.failed	Turn selhal.
item.started / item.updated / item.completed	Tool call / message item.
error	Chyba.

Příklad pipeline:

```

codex exec --json --full-auto "Spusť testy a oprav padající" \
  | jq -c 'select(.type=="item.completed" and .item.kind=="message") | .item'

```

5.6. Resume v CI

```
SESSION=$(codex exec --json --full-auto "Začni implementovat OAuth" \
| jq -r 'select(.type=="thread.started") | .thread.id' | head -1)
```

```
# později
```

```
codex exec --resume "$SESSION" --json --full-auto "Doplň testy"
```

⚠ Přesnou syntax `--resume` u `codex exec` ověřte ve výstupu `codex exec --help` (oficiální dokumentace přidává tento flag postupně). V interaktivním módu je `codex resume` univerzálnější.

6. Slash příkazy v interaktivním TUI

V composer poli napište `/`, zobrazí se nabídka. Stisknutím tabulátoru / šipky vyberete a Enterem potvrdíte.

6.1. Práce s relací

Příkaz	Funkce
<code>/init</code>	Vygeneruje skeleton AGENTS.md v cwd.
<code>/new</code>	Začne novou konverzací v rámci stejné CLI session.
<code>/clear</code>	Vyčistí terminál a začne novou konverzací.
<code>/compact</code>	Komprimuje historii (Codex nabídne shrnutí).
<code>/exit, /quit</code>	Ukončí TUI.
<code>/status</code>	Diagnostika: model, approval policy, writable roots, token usage, git branch.
<code>/statusline</code>	Toggle a řazení status-line itemů v patičce.

6.2. Diff a review

Příkaz	Funkce
<code>/diff</code>	Zobrazí pracovní git diff (workspace-aware, od v0.121+).
<code>/review</code>	Souhrn issues v aktuálním working tree.

6.3. Model, approval, sandbox

Příkaz	Funkce
<code>/model</code>	Picker modelu.
<code>/permissions</code> (alias <code>/approval</code>)	Změna approval modu (Auto, Read Only, ...).
<code>/theme</code>	Picker / preview vizuálního tématu (uloží do <code>[tui].theme</code>).

6.4. Memories, agenti, skills, hooks, plugins

Příkaz	Funkce
/agent	Picker vláken / sub-agentů; přepíná aktivní vlákno (multi-agent).
/skills	Seznam / správa skills. Skill se invokes psaním \$skill-name v promptu.
/hooks	Browser hooků; toggle (od v0.121+).
/mcp	Seznam dostupných MCP nástrojů a status serverů.
/ide	Inject IDE kontextu (od v0.121+).
/mention	Přidá soubor do konverzace s persistentní referencí.

6.5. Vstupní metody, klávesy

Příkaz	Funkce
/keymap	Inspekce a editace TUI klávesových zkratk.
/vim	Modal Vim editing v composeru.
/personality	Změna stylu komunikace (friendly, pragmatic, none).

6.6. Fast mode a feedback

Příkaz	Funkce
/fast	/fast on, /fast off, /fast status (vyžaduje ChatGPT login).
/feedback	Sběr diagnostiky a odeslání týmu OpenAI.
/logout	Odhlášení a smazání credentials.

6.7. (Deprecated) Custom prompts

Příkaz	Funkce
/prompts:<name> [arg1 arg2 ...]	Spuštění uživatelského promptu z ~/ .codex/prompts/<name>.md. (V novějších verzích se preferují skills.)

6.8. Příklady použití

Rychlá orientace v repu:

```
/diff
/review
```

Změna chování za běhu:

```
/model gpt-5-pro
/permissions never
/agent worker
```

Přidání souboru ke kontextu:

```
/mention src/Services/OrderService.cs
```

7. Klávesové zkratky a Vim mode

7.1. Globální zkratky v TUI

Klávesa	Účinek
Ctrl+C	Cancel aktuálního turnu (přerušení generace).
Ctrl+D	Exit TUI.
Ctrl+R	Reverse history search (od v0.121+).
Ctrl+G	Otevře externí editor (\$VISUAL, jinak \$EDITOR).
Ctrl+O	Kopíruje poslední odpověď.
Alt+, / Alt+.	Sníží / zvýší reasoning depth pro další turn.
Shift+Enter	Nový řádek bez odeslání (závisí na terminálu / /keymap).
Esc	Přerušení aktuální editace / akce.
/	Otevře slash menu.
\$	Otevře skill picker (\$skill-name).
@	Mention / reference souboru.

7.2. Vim mode

```
/vim zapne modal editing:
```

- Esc → NORMAL.
- i , a , o , O → INSERT.
- dd , yy , p , u , Ctrl+R (redo).
- : → command-line (např. :wq odešle, :q ukončí composer).

7.3. Vlastní keymap

```
/keymap otevře editor ~/ .codex/keymap.toml (název může být ve formátu
keymap.json / keymap.toml dle verze; ověřte ve výstupu /keymap ).
```

8. Konfigurační hierarchie a `config.toml`

8.1. Lokace

Scope	Cesta
Global	<code>\$CODEX_HOME/config.toml</code> (default <code>~/.codex/config.toml</code>)
Project	<code><projekt>/.codex/config.toml</code> (jen pokud je projekt trusted)
Inline overrides	<code>-c key=val</code>
Profil	<code>[profiles.<name>]</code> v jakémkoliv <code>config.toml</code>
Org-level requirements	<code>requirements.toml</code> (vynucování constraints)

8.2. Hlavní klíče v rootu

```
# Model & reasoning
model = "gpt-5-codex"
model_provider = "openai"
model_reasoning_effort = "medium"           # none|minimal|low|medium|high|xhigh
model_reasoning_summary = "auto"            # auto|concise|detailed|none
model_verbosity = "medium"                  # low|medium|high (Responses API
only)
model_supports_reasoning_summaries = true
model_context_window = 200000
model_max_output_tokens = 100000

# Approval & sandbox
approval_policy = "on-request"
sandbox_mode = "workspace-write"

# Web search
web_search = "cached"                       # cached|live|disabled

# Notifikace
notify = ["/bin/bash", "/Users/me/.codex/hooks/notify.sh"]

# Storage
disable_response_storage = false            # true pro Zero Data Retention účty

# AGENTS.md
project_doc_max_bytes = 65536
```

```
project_doc_fallback_filenames = ["CLAUDE.md", "GEMINI.md"]

# Auth
preferred_auth_method = "apikey"           # apikey|chatgpt
```

8.3. Granular approval policy

Pro precizní řízení per-kategorie:

```
approval_policy = { granular = {
    sandbox_approval = true,
    rules = true,
    mcp_elicitations = true,
    request_permissions = false,
    skill_approval = false
} }
```

8.4. [features]

Přepínače volitelných funkcí (sady se mění verzi od verze – zde výběr aktuálně dokumentovaných):

```
[features]
shell_tool = true
apps = true
codex_hooks = true
unified_exec = true
shell_snapshot = false
multi_agent = true
personality = true
fast_mode = false
enable_request_compression = true
skill_mcp_dependency_install = true
memories = true
experimental_use_rmcp_client = true    # streamable HTTP MCP klient
```

8.5. [sandbox_workspace_write]

```
[sandbox_workspace_write]
writable_roots = ["/Users/me/.pyenv/shims"]
network_access = false
```



```
exclude_tmpdir_env_var = false
exclude_slash_tmp = false
```

8.6. [shell_environment_policy]

Filtrování proměnných předávaných do `shell` toolu:

```
[shell_environment_policy]
inherit = "core"                # all | core | none
ignore_default_excludes = false
exclude = ["AWS_", "AZURE_"]
include_only = ["PATH", "HOME"]
set = { PATH = "/usr/bin", MY_FLAG = "1" }
```

⚠ Default excluduje cokoli s `*KEY*`, `*SECRET*`, `*TOKEN*` v názvu. Pokud vědomě potřebujete `GITHUB_TOKEN` v shellu, explicitně přidejte do `include_only`.

8.7. [history]

```
[history]
persistence = "save-all"      # save-all (default) | none
max_bytes = 10000000           # cap velikost souboru (drop nejstarších při
překročení)
```

8.8. [tui]

```
[tui]
theme = "dracula"
notifications = true
notification_method = "auto"    # auto|osc9|be1
notification_condition = "unfocused" # unfocused|always
animations = true
show_tooltips = true
```

8.9. [memories]

```
[memories]
generate_memories = true
```

```
use_memories = true
disable_on_external_context = true           # vyloučí thready s MCP/web search
```

Memories soubory jsou v `~/codex/memories/` . Detail v kapitole 23.

8.10. [agents]

```
[agents]
max_threads = 4
max_depth = 2
job_max_runtime_seconds = 600
```

Custom agenty se definují v `.codex/agents/<name>.toml` . Detail v kapitole 17.

8.11. Trust projektů

```
[projects."/Users/me/work/safe-repo"]
trust_level = "trusted"           # trusted|untrusted

[projects."/Users/me/work/risky"]
trust_level = "untrusted"
```

 Pouze v trusted projektech se aplikuje `.codex/config.toml` , `.codex/agents/` , `.codex/skills/` , `.codex/hooks/` a project-scope MCP servery.

8.12. JSON-Schema validace

Codex zatím nedistribuuje veřejné JSON Schema pro `config.toml` , ale validace TOML se řeší pluginy editoru (Even Better TOML pro VS Code, native podpora v IntelliJ Rust pluginu). Komunita publikuje schémata na GitHub gitech.

9. Profily ([profiles.*])

Profil = pojmenovaná sada klíčů, kterou aktivujete jedním flagem `--profile` . Skvělé pro:

- různé modely v dev / review / CI;
- různé providery (OpenAI vs Azure vs Ollama);
- různé sandbox / approval politiky.

9.1. Příklady

```
[profiles.development]
model = "gpt-5.1-codex"
```

```
approval_policy = "on-request"
sandbox_mode = "workspace-write"
model_reasoning_effort = "medium"
model_verbosity = "medium"
```

[profiles.review]

```
model = "gpt-5-pro"
model_reasoning_effort = "high"
approval_policy = "never"
sandbox_mode = "read-only"
```

[profiles.ci]

```
approval_policy = "never"
sandbox_mode = "danger-full-access"
model_reasoning_effort = "low"
```

[profiles.openrouter]

```
model = "anthropic/claude-3.5-sonnet"
model_provider = "openrouter"
```

[profiles.azure]

```
model = "gpt-4.1"
model_provider = "azure"
```

9.2. Aktivace

```
codex --profile development
codex exec --profile review "Code review of base-branch main" --json
```

9.3. Skládání

CLI flagy stále přebíjí profil:

```
codex --profile development -m gpt-5-pro --ask-for-approval never
```

10. Modely a model providers

10.1. Defaultní model

Aktuální verze (v0.130) jako default obvykle volí **gpt-5-codex** (resp. specializovaný coding model dané OpenAI generace). Default se mění s release; proto **nikdy nehardcodejte v dokumentaci verzi modelu** – odkazujte na `/status` a

```
codex --help .
```

10.2. Wire API

```
[model_providers.openai]
wire_api = "responses"    # OpenAI Responses API (s reasoning summaries)

[model_providers.openrouter]
wire_api = "chat"         # klasické chat completions
```

Volba ovlivňuje, zda lze používat reasoning summaries, prompt caching a další pokročilé funkce.

10.3. Vestavěné providery (typické)

```
[model_providers.openai]
name = "OpenAI"
base_url = "https://api.openai.com/v1"
wire_api = "responses"
env_key = "OPENAI_API_KEY"

[model_providers.azure]
name = "Azure OpenAI"
base_url = "https://YOUR.openai.azure.com/openai"
wire_api = "responses"
query_params = { "api-version" = "2025-04-01-preview" }
env_key = "AZURE_OPENAI_API_KEY"
env_http_headers = {
  "OpenAI-Organization" = "OPENAI_ORGANIZATION",
  "OpenAI-Project" = "OPENAI_PROJECT"
}
http_headers = { "X-Internal" = "static-value" }
request_max_retries = 4
stream_max_retries = 5
stream_idle_timeout_ms = 300000

[model_providers.openrouter]
name = "OpenRouter"
base_url = "https://openrouter.ai/api/v1"
wire_api = "chat"
env_key = "OPENROUTER_API_KEY"

[model_providers.oss]
```

```
name = "Local Ollama"
base_url = "http://localhost:11434/v1"
wire_api = "chat"
```

[model_providers.lmstudio]

```
name = "LM Studio"
base_url = "http://localhost:1234/v1"
wire_api = "chat"
```

10.4. Klíče provideru

Klíč	Význam
name	Lidsky čitelné jméno.
base_url	API endpoint.
wire_api	responses / chat.
env_key	Název env proměnné s API klíčem.
query_params	Query parametry doplněné ke každému requestu (Azure api-version).
http_headers	Statické HTTP hlavičky.
env_http_headers	Mapa header → env var (hodnota se vyzvedne z env při běhu).
request_max_retries	Počet retry při 5xx.
stream_max_retries	Retry pro stream rozbité během odpovědi.
stream_idle_timeout_ms	Idle timeout streamu.

10.5. Alternativní providery

Codex obecně podporuje libovolný provider implementující OpenAI Chat / Responses API:

- **Mistral** (la Plateforme),
- **Together AI**,
- **Groq**,
- **Ollama** (lokálně, OpenAI-compatible endpoint),
- **LM Studio**,
- **Morph**,
- **Cerebras Inference**.

Pro tyto stačí definovat `[model_providers.<name>]` a volat `--profile / -c model_provider="<name>"`.

10.6. Spuštění s lokálním modelem

```
ollama serve
ollama pull qwen3-coder:30b
codex --oss -m qwen3-coder:30b
```

 Lokální modely běží *bez* prompt cachingu OpenAI a obvykle bez tool callingu na úrovni pokročilých modelů. Kontext je omezenější – očekávejte jiný workflow než s `gpt-5-codex` .

11. Reasoning, verbosity, web search

11.1. Reasoning effort

Hodnota	Význam
none	Bez explicitního reasoningu. Levné a rychlé, vhodné pro lint-like úkoly.
minimal	Jen minimální plánování.
low	Krátké reasoning summaries.
medium	Default; rozumný kompromis pro většinu coding úloh.
high	Hluboká analýza, vhodné pro architekturu, security review.
xhigh	Extrémní hloubka. Drahé; jen pro kritická rozhodnutí.

```
model_reasoning_effort = "high"
```

11.2. Reasoning summary

Hodnota	Význam
auto	Codex se rozhodne podle úkolu.
concise	Stručný souhrn rozhodnutí.
detailed	Detailní souhrn (drahé na tokeny).
none	Bez summary.

11.3. Verbosity

Hodnota	Význam
low	Krátké odpovědi.
medium	Default.
high	Detailní odpovědi.

⚠ Verbose má vliv jen u Responses API (`wire_api = "responses"`).

11.4. Web search

Hodnota	Význam
cached	Default; využívá indexovanou cache OpenAI.
live	Live fetch URL (drahé na tokeny, pomalejší). Ekvivalent CLI flag <code>--search</code> .
disabled	Vypnuto.

```
web_search = "cached"
```

11.5. Live web search ad hoc

```
codex --search "Najdi mi nejnovější release notes EF Core 9"
```

V interaktivním TUI:

```
> /enable web_search live
```

(Přesnou syntax ověřte v `--help` ; aktuálně lze použít `-c web_search='"live"'` .)

12. AGENTS.md – paměť projektu

AGENTS.md je markdown soubor s persistentními instrukcemi pro agenta. Codex ho čte při startu relace a vkládá do system promptu. Funkčně analogický **CLAUDE.md** .

12.1. Lookup hierarchie

1. Global scope:

- `$CODEX_HOME/AGENTS.override.md` (přepíše níže), jinak
- `$CODEX_HOME/AGENTS.md` .

2. Project scope: Codex prochází od kořene projektu (typicky git root) směrem ke cwd. V každém adresáři kontroluje:

- `AGENTS.override.md` ,
- `AGENTS.md` ,
- fallback jména z `project_doc_fallback_filenames` (např. `CLAUDE.md` , `GEMINI.md`). Hledání končí v cwd.

3. Všechny nalezené soubory se konkatenují, ohraničené hlavičkami.

12.2. Doporučená šablona

AGENTS.md

Kontext projektu

Knihovna pro generování CLI fasád v .NET 9. Cílová publika: backend týmy uvnitř firmy.

Tech stack

- .NET 9, C# 13, nullable reference types ON
- xUnit + FluentAssertions + Moq
- Spectre.Console pro UI vrstvu
- GitHub Actions pro CI

Konvence

- Async metody mají suffix ``Async`` a vždy přijímají ``CancellationToken``.
- DI registrace v ``ServiceCollectionExtensions``, jméno ``Add{Feature}``.
- Logování přes ``ILogger<T>`` se source-generated metodami (``LoggerMessage``).
- Žádné ``var`` v public API – preferujeme explicitní typy.

Build & test

```
dotnet restore dotnet build --no-restore dotnet test --collect:"XPlat Code Coverage"
```

Architektonická pravidla

- Žádný kód nezávisí na konkrétním ORM.
- Konfigurace přes ``IOptions<T>``, nikdy přímo ``IConfiguration`` v service vrstvě.

Co nedělat

- Neměnit veřejné API bez bumpnutí major verze.
- Neukládat tajemství do ``appsettings*.json``.

12.3. Limit `project_doc_max_bytes`

Defaultní limit (~64 KiB v aktuálních verzích) ořezává AGENTS.md při překročení. ⚠ Příliš dlouhé soubory se *tiše* truncatují, takže obsah na konci se nemusí dostat do kontextu. Známy issue #7138 v repu.

12.4. Globální vs project AGENTS.md

- **Global** (`~/ .codex/AGENTS.md`): osobní preference (jazyk odpovědí, oblíbený formátovač, no-emoji policy).
- **Project**: konvence týmu, tech stack, build & test příkazy.

12.5. Override mechanismus

Soubor `AGENTS.override.md` (v daném scope) nahradí odpovídající `AGENTS.md`. Užitečné, když chcete experimentovat s instrukcemi, aniž byste přepsali týmový soubor.

13. Approval policy a sandbox modes

13.1. Approval modes

Mode	Chování
untrusted	Známé safe operace bez ptaní; mutace / destruktivní příkazy se ptají.
on-request	Codex se ptá vždy, když potřebuje něco mimo sandbox (default v interaktivním).
never	Žádná schválení; kombinujte se sandboxem a/nebo Dockerem.
granular	Per-kategorie (viz 8.3).

⚠ Hodnota `on-failure` je **deprecated** a v nových verzích neaktivní.

13.2. Sandbox modes

Mode	Co povoluje
read-only	Pouze čtení; žádné writes ani exec mimo bezpečné.
workspace-write	Write v cwd a writable_roots, read-only mimo.
danger-full-access	Bez sandboxu (Docker / VM). Opt-in.

13.3. Vzájemné kombinace

```
# Bezpečný onboarding nového repozitáře
approval_policy = "untrusted"
sandbox_mode = "read-only"

# Standardní vývoj
approval_policy = "on-request"
sandbox_mode = "workspace-write"

# CI v Dockeru
approval_policy = "never"
sandbox_mode = "danger-full-access"
```

13.4. Granular policy

```

approval_policy = { granular = {
    sandbox_approval = true,      # ptát se při sandbox výjimkách
    rules = true,                # ptát se při porušení rules
    mcp_elicitations = true,     # ptát se na MCP elicítace
    request_permissions = false, # neptát se při dotazu agenta na další
oprávnění
    skill_approval = false       # neptát se před spuštěním skillu
} }

```

13.5. Organizační constraints (`requirements.toml`)

Organizace mohou centrálně blokovat nebezpečné kombinace:

```

# requirements.toml v org-level distribuci
disallow_approval_policy = ["never"]
disallow_sandbox_mode = ["danger-full-access"]

```

Pokud uživatel zkusí nasadit zakázanou kombinaci, Codex spuštění odmítne s explicitní chybou.

14. Sandboxing detailně

14.1. macOS – Apple Seatbelt

Codex generuje Seatbelt profil podle modu a předá `sandbox-exec` . Profil omezuje:

- write na `cwd` + `writable_roots` (workspace-write),
- network (povoleno jen v `danger-full-access`),
- spawn neznámých binárek (filtruje per-mode).

Manuální použití:

```

codex sandbox seatbelt -- /bin/bash -c 'echo hello && curl
https://example.com'

```

14.2. Linux – Landlock + seccomp

Codex postaví Landlock ruleset:

- `LANDLOCK_ACCESS_FS_WRITE_FILE` / `READ_FILE` pro povolené cesty,
- seccomp blokuje rizikové syscalls (`mount` , `clone3` s neopálenými flagy, `bpf` , ...).

Manuální:

```
codex sandbox landlock -- cargo build
```

⚠ Landlock vyžaduje Linux ≥ 5.13 s povoleným ABI 2+. Na starších kernelech Codex spadne zpět na soft-restriction (`approval_policy` + filesystem checks v Rust kódu).

14.3. Windows

Windows nemá ekvivalent Landlocku. Codex v0.130+ přidal hardening:

- ACL kontroly (write jen na cwd a explicitní `writable_roots`),
- network filter na úrovni runtime kontroly před fetch / exec,
- WSL2 doporučeno pro „pravý“ sandbox – uvnitř WSL platí Linuxový Landlock+seccomp.

14.4. Docker / kontejnery

Doporučený pattern v CI:

```
FROM openai/codex-universal:latest

RUN codex login --with-api-key < /run/secrets/openai_api_key
ENV CODEX_HOME=/root/.codex

# vstupní bod
CMD ["codex", "exec", "--full-auto", "--sandbox", "danger-full-access", \
    "--ask-for-approval", "never", "Run unit tests and fix failures"]
```

🛡 Sandboxy uvnitř kontejneru nečtou user-level config (`~/codex` mimo kontejner). Project-level (`.codex/config.toml`) ale ano, pokud je projekt namountovaný.

14.5. `[sandbox_workspace_write]` detailně

```
[sandbox_workspace_write]
writable_roots = [
    "/Users/me/.cache",
    "/Users/me/.pyenv/shims"
]
network_access = false
exclude_tmpdir_env_var = false      # nezohledňovat $TMPDIR jako writable
exclude_slash_tmp = false           # nezohledňovat /tmp jako writable
```

`network_access = true` v `workspace-write` povoluje TCP / DNS – vyplatí se např. pro `npm install`, `dotnet restore`. Ale stále neumožňuje SSH apod.

14.6. `exec_command_timeout_ms`

```
exec_command_timeout_ms = 60000    # 60 s
```

Globální timeout pro shell exec calls.

14.7. `experimental_codex_linux_sandbox_exe`

```
experimental_codex_linux_sandbox_exe = "/usr/local/bin/codex-linux-sandbox"
```

Override binárky, která realizuje Landlock+seccomp. Užitečné, když je Codex zabalen v setuid binárce nebo když integrujete vlastní sandbox runner.

15. Environment policy a shell prostředí

15.1. `[shell_environment_policy]`

Default Codex z bezpečnostních důvodů odfiltruje proměnné s názvy obsahujícími `KEY`, `SECRET`, `TOKEN`, `PASSWORD` apod. Politiku lze nastavit takto:

```
[shell_environment_policy]
inherit = "core"                # all | core | none
ignore_default_excludes = false
exclude = ["AWS_*", "AZURE_*"]
include_only = ["PATH", "HOME", "LANG", "LC_*"]
set = { CI = "1", DOTNET_NOLOGO = "1" }
```

Sémantika:

Klíč	Význam
<code>inherit</code>	<code>all</code> (vše), <code>core</code> (jen základní – <code>PATH</code> , <code>HOME</code> , <code>USER</code> , <code>LANG</code> , <code>LC_*</code> , ...), <code>none</code> (nic).
<code>ignore_default_excludes</code>	Pokud <code>true</code> , neaplikovat default filtr na <code>*KEY*/*SECRET*/*TOKEN*</code> .  S rozvahou.
<code>exclude</code>	Glob patterny pro odstranění.
<code>include_only</code>	Pokud uvedeno, pouze proměnné z této množiny projdou.
<code>set</code>	Hard-coded hodnoty (přepíše <code>inherit</code>).

15.2. Doporučené konfigurace

Bezpečná default-firmová:

```
[shell_environment_policy]
inherit = "core"
exclude = ["AWS_*", "AZURE_*", "GCP_*", "GITHUB_TOKEN"]
set = { CI = "1" }
```

Pro lokální dev s povolením GitHub tokenu:

```
[shell_environment_policy]
inherit = "core"
include_only = ["PATH", "HOME", "USER", "LANG", "LC_*", "GITHUB_TOKEN"]
```

16. Vestavěné nástroje (Tools)

Codex má v jádru sadu built-in toolů, které model volá podle potřeby. Tato kapitola je referenční – samotné rozhodování o použití je v rukou modelu.

16.1. Shell / exec

Výchozí tool pro spouštění shellových příkazů. Respektuje `shell_environment_policy`, `sandbox` a `approval policy`.

- `unified_exec = true` v `[features]` zapíná novější runner s lepší podporou interaktivních příkazů.
- Timeout řízen `exec_command_timeout_ms`.

16.2. File tools

- **Read** – čtení souborů (text + obrázky + PDF).
- **Write / Apply patch** – Codex preferuje generování `apply_patch` formátu (unified diff podobný), který umí aplikovat atomicky.
- **List / search** – `ls`, `find`, `grep`, případně `rg` (přes shell tool).

16.3. View image

`view_image` tool načte přiložený obrázek. Od v0.117 vrací URLs v code mode (lze odkazovat z odpovědi).

16.4. Plan tool

S `--include-plan-tool` v `exec` módu nebo v některých interaktivních scénářích. Slouží k explicitnímu plánování multi-step úkolu.

16.5. Web search

Pokud `web_search != "disabled"`, model může volat web search nástroj.

16.6. MCP tools

Externí nástroje vystavené MCP servery se zobrazují modelu jako tools s prefixem `mcp_<server>__<tool>` .
Detail v kapitole 20.

16.7. Apps (od v0.121+)

`apps = true` v `[features]` aktivuje „Apps“ – pojmenované, předem definované balíčky konfigurace (typicky pro výrobu pluginů/skillů).

16.8. Skills

Skills nejsou „tool“ v klasickém smyslu, ale jejich invokace `$skill-name` se chová podobně jako tool call. Detail v kapitole 18.

17. Subagenti (multi-agent, threads)

Codex podporuje **vícevláknový agentní běh** – multi-agent. Stejný TUI udržuje N paralelních threadů, mezi kterými se přepíná přes `/agent` .

17.1. Konfigurace v rootu

```
[agents]
max_threads = 4
max_depth = 2
job_max_runtime_seconds = 600
```

Klíč	Význam
max_threads	Maximální počet paralelních threadů.
max_depth	Hloubka delegace (jak hluboko může agent vytvářet subagenty).
job_max_runtime_seconds	Timeout subjobu.

17.2. Built-in agenty

- Codex má vestavěné role:
- `default` – hlavní orchestrátor.
 - `worker` – exekuční agent pro konkrétní úkoly.
 - `explorer` – průzkumný agent (read-only sandbox preferred).

17.3. Custom subagent v TOML

`.codex/agents/security-reviewer.toml` :

```

name = "security-reviewer"
description = "Provádí security audit (SQL injection, XSS, auth bypass).\"
developer_instructions = \"\"\"
Jsi expert na bezpečnost webových aplikací.

## Postup
1. Identifikuj všechny vstupní body (controllery, API endpointy,
deserializace).
2. Vyhodnoť každý vstup proti OWASP Top 10.
3. Označ rizikové kódy a navrhní opravu.

## Co NE
- Neměň produkční kód, jen reportuj.
- Nepiš testy (to dělá jiný agent).
\"\"\"
model = "gpt-5-pro"
sandbox_mode = "read-only"
approval_policy = "never"
model_reasoning_effort = "high"

```

17.4. Volání agenta

V interaktivním TUI:

```
/agent security-reviewer
```

Nebo explicitní v promptu:

```
„Use the security-reviewer agent to audit Services/Auth.“
```

17.5. Multi-thread workflow

Příklad: hlavní agent zadá worker thread na implementaci, druhý explorer thread na průzkum dependencies, třetí review thread na kontrolu na konci. Mezi nimi se přepíná `/agent` .

```

[default]    > Naplánuj migraci DataAccess vrstvy.
[plan se zobrazí]
[default]    > /agent worker
[worker]     > Implementuj kroky 1–3.
[worker]     > /agent explorer
[explorer]   > Vyjmenuj všechny ORM volání mimo Repository třídy.
[explorer]   > /agent default
[default]    > Spust review.

```

17.6. Hloubka subagentů

`max_depth = 2` znamená: hlavní agent → worker → ne dále. Tím se předchází nekonečnému řetězení, které by mohlo eskalovat náklady.

18. Skills a (deprecated) custom prompts

Skills jsou novější formát rozšíření – analogie Anthropic skills. Custom prompts (v `~/.codex/prompts/*.md`) jsou starší a postupně se migrují na skills.

18.1. Skills – lokace a struktura

Scope	Cesta
User	<code>~/.codex/skills/<name>/</code>
Project	<code>.codex/skills/<name>/</code>
Plugin	uvnitř pluginu

Adresář obsahuje:

- `SKILL.md` (povinné) – frontmatter + instrukce,
- `references/`, `scripts/` (volitelné) – on-demand soubory.

18.2. Frontmatter `SKILL.md`

```
---
name: ef-core-migrations
description: Generuje a kontroluje EF Core migrace v .NET projektech.
Spouštěj při zmínkách "migrace", "DbContext", "Add-Migration".
---
```

Jsi expert na EF Core 9 migrace v .NET 9.

Postup

1. Identifikuj DbContext.
2. Zkontroluj pending změny (``dotnet ef migrations list``).
3. Navrhni jméno migrace v PascalCase.
4. Spuštění ``dotnet ef migrations add <Name>``.
5. Verifikuj Up/Down a označ destruktivní operace.

Reference

@references/ef-style-guide.md

18.3. Invokace

- **Explicitní:** v promptu napište `$ef-core-migrations` → Codex skill načte a aplikuje.
- **Implicitní:** Codex se podle popisu rozhodne sám.

18.4. (Deprecated) custom prompts

`~/ .codex/prompts/<name>.md :`

```
---
description: Vygeneruj Conventional Commit zprávu.
argument-hint: [scope]
---

Načti `git diff --staged` a vrať Conventional Commit zprávu se scopem **$1**.
```

Spuštění: `/prompts:<name> auth .` Placeholdery `$1 .. $9` a `$NAME` .

⚠ V nových verzích (od v0.121+) se preferují skills – funkčně silnější (mohou přiložit reference soubory, scripty, mít `skill_approval`, apod.).

18.5. Skill MCP dependency install

v `[features]` :

```
skill_mcp_dependency_install = true
```

Skill může deklarovat MCP servery, které potřebuje, a Codex je při invokaci zavede do session (pokud není uživatel v sandboxu, který by to blokoval).

19. Hooks

Hooks jsou shell skripty napojené na lifecycle eventy. Slouží k auditu, vynucování pravidel a integracím (Slack, GitHub).

19.1. Lifecycle eventy

Event	Kdy se spustí
PreToolUse	Před voláním toolu (od v0.121+ s rozšířeným kontextem).
PostToolUse	Po úspěšném tool callu.
PermissionRequest	Při žádosti o oprávnění.
UserPromptSubmit	Když uživatel odešle prompt.
Stop	Když Codex dokončí turn.

PreCompact	Před /compact.
PostCompact	Po /compact.

⚠ Některé hooky neaktivují u `apply_patch` a MCP tool calls – sledujte changelog ke konkrétní verzi.

19.2. Konfigurace hooks

Hooks lze definovat:

- centrálně v `~/.codex/hooks/hooks.json` (resp. `.codex/hooks/hooks.json` pro project),
- nebo bundle v pluginu (`.codex-plugin/hooks/`).

```
{
  "PreToolUse": [
    { "matcher": "shell", "command": [".codex/hooks/audit-shell.sh"] }
  ],
  "PostToolUse": [
    {
      "matcher": "apply_patch",
      "command": ["dotnet", "format", "--include", "$CODEX_PATCH_FILES"]
    }
  ],
  "UserPromptSubmit": [
    { "command": [".codex/hooks/log-prompt.sh"] }
  ],
  "Stop": [
    { "command": [".codex/hooks/notify-slack.sh"] }
  ]
}
```

(Detailní syntax je závislá na verzi – v `/hooks` v TUI najdete platnou strukturu pro vaši instalaci.)

19.3. Vstup hooku

Hook obdrží JSON na stdin s eventem (tool name, parametry, soubor, prompt). Vráť JSON na stdout, případně non-zero exit code = block.

```
{ "decision": "allow", "reason": "audit logged" }
```

19.4. Užitečné env proměnné v hooku

Proměnná	Význam
----------	--------

CODEX_SESSION_ID	UUID relace.
CODEX_TOOL	Jméno toolu.
CODEX_CWD	cwd.
CODEX_PATCH_FILES	Seznam souborů z apply_patch.

(Konkrétní názvy proměnných se v různých verzích mírně liší – ověřte `codex --help` / `/hooks` .)

19.5. Příklady

Audit shell calls:

```
#!/usr/bin/env bash
read -r INPUT
echo "$(date -Iseconds) $CODEX_SESSION_ID $INPUT" >> ~/.codex/audit.log
echo '{"decision":"allow"}'
```

Format po patchi:

```
#!/usr/bin/env bash
read -r INPUT
for FILE in $CODEX_PATCH_FILES; do
  case "$FILE" in
    *.cs)          dotnet format --include "$FILE" ;;
    *.ts|*.tsx)    npx prettier --write "$FILE" ;;
  esac
done
echo '{"decision":"allow"}'
```

Slack notifikace na konci turnu:

```
#!/usr/bin/env bash
curl -s -X POST "$SLACK_WEBHOOK" -H "Content-Type: application/json" \
  -d '{"text": "\nCodex finished turn in $(basename $CODEX_CWD)\n"}'
echo '{"decision":"allow"}'
```

19.6. /hooks

V TUI `/hooks` zobrazí browser nadefinovaných hooků a umožní je toggle pro aktuální session.

20. MCP integrace

Codex má vestavěného MCP klienta (stdio default; streamable HTTP přes `experimental_use_rmcp_client`).

20.1. Stdio servery

```
[mcp_servers.docs]
command = "node"
args = ["/mcp/docs-server.mjs"]
env = { LOG_LEVEL = "info" }
env_vars = ["DOCS_TOKEN"]           # whitelist env var z parent shellu
startup_timeout_sec = 10
tool_timeout_sec = 60
required = true
experimental_environment = "remote" # remote executor (volitelně)
```

Klíče:

Klíč	Význam
command	Spustitelný soubor / interpret.
args	Argumenty.
env	Mapa env proměnných (statické hodnoty).
env_vars	Whitelist názvů env proměnných z parent shellu, které se předají serveru.
startup_timeout_sec	Timeout startu.
tool_timeout_sec	Timeout per-tool call.
required	Pokud true, selhání startu zastaví celou relaci.
experimental_environment	local / remote.

20.2. Streamable HTTP servery

```
[mcp_servers.figma]
url = "http://127.0.0.1:3845/mcp"
bearer_token = "secret"
# nebo z env:
bearer_token_env_var = "FIGMA_TOKEN"
```

Vyžaduje:

[features]

```
experimental_use_rmcp_client = true
```

🛡️ Tokeny vždy přes `bearer_token_env_var`, nikdy literálně do souboru pod git.

20.3. CLI správa MCP serverů

```
codex mcp list
codex mcp add docs -- node ./mcp/docs-server.mjs
codex mcp add figma --url http://127.0.0.1:3845/mcp \
  --bearer-token-env-var FIGMA_TOKEN
codex mcp login figma          # OAuth pro streamable HTTP servery
codex mcp remove docs
```

20.4. Codex jako MCP server

```
codex mcp-server
```

Codex sám exponuje vlastní funkcionalitu jako MCP server, takže ho může konzumovat jiný agent (Claude, IDE plugin).

20.5. Použití v relaci

V interaktivním TUI:

```
/mcp          # status, dostupné nástroje
```

V promptu:

```
„Použij MCP nástroj docs.search k vyhledání článku o EF Core.“
```

20.6. Debug

```
RUST_LOG=mcp=debug codex
```

V TUI: `/mcp` ukáže poslední chyby a možnost restartu serveru.

20.7. Sdílení MCP konfigurace s VS Code rozšířením

OpenAI VS Code extension čte stejné `mcp_servers` z `~/.codex/config.toml`. Není nutná samostatná konfigurace – co nastavíte v CLI, vidí i IDE.

21. Plugins a marketplaces

21.1. Pluginy

Plugin je distribuovatelný balíček obsahující kombinaci skills, hooks, MCP serverů a apps.

Manifest v `.codex-plugin/plugin.json` :

```
{
  "name": "csharp-helpers",
  "version": "0.2.0",
  "description": "Sada pomocníků pro C#/.NET týmy.",
  "author": "Acme Corp",
  "homepage": "https://github.com/acme/csharp-helpers",
  "repository": "https://github.com/acme/csharp-helpers",
  "license": "MIT",
  "keywords": ["csharp", "dotnet", "ef-core"],
  "skills": ["./.skills/ef-core-migrations", "./.skills/scaffold-test"],
  "mcpServers": {
    "csharp-docs": {
      "command": "node",
      "args": ["./.mcp/csharp-docs.mjs"]
    }
  },
  "hooks": {
    "PostToolUse": [
      { "matcher": "apply_patch", "command": ["dotnet", "format"] }
    ]
  }
}
```

21.2. Marketplaces

Marketplace = git repozitář / lokální adresář s indexem pluginů.

```
# přidání oficiálního marketplace
codex plugin marketplace add openai/codex-plugins

# soukromý firemní marketplace
codex plugin marketplace add github.com/acme/codex-plugins --ref main

# lokální cesta pro vývoj
codex plugin marketplace add ./local-marketplace-root
```

```
# sparse checkout subdirectory
codex plugin marketplace add https://github.com/example/plugins.git --sparse
.agents/plugins

# upgrade všech
codex plugin marketplace upgrade

# odstranění
codex plugin marketplace remove csharp-helpers
```

21.3. Instalace pluginu

```
codex plugin install csharp-helpers
codex plugin list
codex plugin remove csharp-helpers
```

21.4. Plugin v projektu vs uživateli

Pluginy lze instalovat:

- globálně (`codex plugin install` bez specifikace → user scope),
- per-projekt (config v `.codex/plugins/` při trusted projektu).

22. Notifikace

22.1. Externí program přes `notify`

```
notify = ["/bin/bash", "/Users/me/.codex/hooks/notify.sh"]
```

Skriptu se předává JSON jako argv (typicky `argv[1]`); aktuálně podporovaná událost: `agent-turn-complete`.

```
#!/usr/bin/env bash
JSON="$1"
TYPE=$(echo "$JSON" | jq -r .type)

if [[ "$TYPE" == "agent-turn-complete" ]]; then
  osascript -e 'display notification "Codex finished" with title "Codex"'
fi
```

(Linux: `notify-send` . Windows: PowerShell `New-BurntToastNotification` z modulu `BurntToast`.)

22.2. TUI notifikace

```
[tui]
notifications = true                # nebo array typů událostí
notification_method = "auto"       # auto | osc9 | bel
notification_condition = "unfocused" # unfocused | always
```

Metoda	Význam
auto	Vyber dle terminálu.
osc9	OSC9 escape sekvence (moderní terminály – iTerm2, Windows Terminal).
bel	ASCII BEL (kompatibilní, jen zvuk).

`notification_condition = "unfocused"` upozorňuje jen když okno terminálu není v popředí.

23. Memories (perzistentní paměť)

Od v0.100 Codex umí ukládat „memories” – strukturované poznámky, které generuje sám během práce a re-loaduje v dalších relacích.

23.1. Konfigurace

```
[memories]
generate_memories = true
use_memories = true
disable_on_external_context = true
```

Klíč	Význam
generate_memories	Codex vytváří memory záznamy.
use_memories	Předchozí memory se loadují do system promptu.
disable_on_external_context	Pokud thread používá MCP / web search, memories se vypnou (ochrana před exfiltrací do externích serverů).

23.2. Lokace

`~/.codex/memories/` – obvykle JSONL, typicky strukturováno per-projekt.

23.3. Vztah k AGENTS.md

`AGENTS.md` jsou explicitní, ručně psané, deterministické instrukce. Memories jsou implicitní, dynamicky generované, automaticky používané.

🛡️ Memories nejsou commitované do gitu (jsou v `~/.codex/`), takže neopouštějí váš stroj. Pro Zero Data Retention tarify ale doporučujeme `generate_memories = false`, aby se nezapisovaly i sensitivní fragmenty z konverzace.

24. Auth a sessions

24.1. Tři způsoby přihlášení

ChatGPT OAuth (browser)

```
codex login
```

Otevře browser, dokončí OAuth s ChatGPT účtem (Plus/Pro/Team/Enterprise). Tento mód umožňuje **fast mode** (`/fast on`) a má jiné účtování než API.

Device-code (headless)

```
codex login --device-auth
```

Vypíše kód, který zadáte na `https://chat.openai.com/auth/device`. Ideální pro SSH terminály, Docker bez X serveru, CI/CD.

API key

```
echo "sk-..." | codex login --with-api-key  
# nebo:  
export OPENAI_API_KEY="sk-..."
```

`OPENAI_API_KEY` se obvykle aplikuje automaticky. Pro vynucení preference:

```
preferred_auth_method = "apikey"
```

24.2. Storage credentials

`$CODEX_HOME/auth.json` – obsahuje OAuth refresh token nebo API key. Citlivost = úroveň hesla.

🛡️ Nikdy neumísťujte do git repa, do CI artefaktů, ani do veřejně dostupných lokací.

24.3. Vícenásobné účty / role

Codex aktuálně nemá nativní multi-user switching – pro práci s více účty:

- různé `CODEX_HOME` :

```
CODEX_HOME=~/.codex-work codex login
CODEX_HOME=~/.codex-personal codex
```

- nebo různé profily a `OPENAI_API_KEY` per terminál.

24.4. Sessions

Aspect	Hodnota
Lokace transkriptu	<code>\$CODEX_HOME/sessions/YYYY/MM/DD/rollout-*.jsonl</code>
Obsah	Conversation history, tool calls, token usage
Identifikátor	UUID v JSON
Resume	<code>codex resume <SESSION_ID> / --last / --all</code>

24.5. Manuální načtení session

```
codex -c experimental_resume="/Users/me/.codex/sessions/2026/05/10/rollout-1234.jsonl"
```

24.6. Mazání sessions

Sessions se manuálně neudržují – zatím chybí konfigurace na auto-cleanup. Doporučená cron rule:

```
find ~/.codex/sessions -mtime +30 -delete
```

25. App-server, remote-control, IDE integrace

25.1. App-server

Codex jádro lze provozovat jako bidirectional JSON-RPC 2.0 server. Default transport: stdio.

```
codex app-server # stdio
codex app-server proxy # otevře unix socket
codex app-server --listen ws://0.0.0.0:7777
```

Volba	Význam
stdio	Default, ideální pro IDE pluginy.
proxy	Unix socket (\$CODEX_HOME/app-server-control/app-server-control.sock, lze přepsat --sock PATH).
WebSocket	--listen ws://..., experimentální.

 Bez auth funguje WS jen na loopbacku. Pro non-loopback nasad'te HMAC-signed JWT/JWS bearer auth.

25.2. Remote-control endpoint (v0.130+)

```
codex remote-control --listen ws://...
```

Top-level endpoint pro headless remote-control. Zjednodušuje provoz centralizovaného Codex serveru, ke kterému se připojují tenké klienty.

25.3. TUI remote attach

```
codex --remote ws://devbox:7777
```

Otevře plný TUI proti vzdálenému Codex jádru. Praktické v dev clusterech a hosted runners.

25.4. VS Code rozšíření

Aspekt	Detail
Marketplace ID	openai.chatgpt
Chat panel	Vedle editoru, čte open files a selection.
@filename.tsx	Explicitní reference na soubor v workspace.
Mode switcher	Chat (plánování) / Agent (full access).
Cloud delegation	Lokální start → delegate to Codex Cloud → progress v IDE.
Sdílení config	Stejně ~/.codex/config.toml, stejné mcp_servers.

Klávesová zkratka `chatgpt.addToThread` (bindable) přidá selection do dalšího promptu.

25.5. JetBrains

K dispozici je oficiální plugin s podobnou funkcionalitou. Pro Rider (C#) a IntelliJ IDEA. Plugin orchestruje CLI běžící v IDE terminálu.

25.6. MCP server pro jiné agenty

```
codex mcp-server
```

Spustí Codex jako MCP server (stdio) – jiný agent (Claude Code, jiný Codex, vlastní orchestrátor) ho může vidět jako externí tool a delegovat na něj úkoly.

26. Logy, debug, tracing

26.1. Lokace logů

OS	Cesta
macOS / Linux	<code>\$CODEX_HOME/log/codex-tui-*.log</code>
Windows	<code>%CODEX_HOME%\log\codex-tui-*.log</code>

26.2. RUST_LOG

Hodnota	Význam
error	Pouze chyby.
warn	+ varování.
info	Default.
debug	+ diagnostika tool calls, MCP, API.
trace	Maximum (zpomaluje 10–50 %).

```
RUST_LOG=debug codex
RUST_LOG=info,codex_core=debug codex
RUST_LOG=codex_core::agent::process_event=trace codex
```

 **trace** umí zaplnit disk; používejte cíleně a s rotací logů.

26.3. Tail v reálném čase

```
tail -f ~/.codex/log/codex-tui-*.log
```

PowerShell:

```
Get-Content "$env:CODEX_HOME\log\codex-tui-*.log" -Wait -Tail 20
```

26.4. /feedback

V TUI `/feedback` umí přiložit aktuální logy a poslat tým OpenAI. Při problému s Codex CLI je to nejjednodušší cesta.

26.5. OpenTelemetry

V0.130+ podporuje konfigurovatelné OpenTelemetry trace metadata. Detail v `[tui] / [telemetry]` sekci v `config.toml` (sekce se vyvíjí; ověřte `codex --help`).

26.6. Network debugging

```
RUST_LOG=hyper=debug,request=debug codex
```

Zobrazí HTTP komunikaci s providerem.

27. Cost a usage tracking

27.1. /status

V TUI `/status` zobrazí mj. token usage aktuální relace:

```
Model:      gpt-5-codex
Provider:    openai
Approval:    on-request
Sandbox:     workspace-write
Web search:  cached
Working dir: /Users/me/work/repo-a (clean)
Tokens:      12 480 in / 4 220 out / 8 192 cached
```

27.2. --json v exec módu

```
codex exec --json --full-auto "..." | jq -c 'select(.type=="turn.completed")
| .usage'
```

Výstup:

```
{ "input_tokens": 4231, "cached_input_tokens": 4203, "output_tokens": 908 }
```

27.3. Optimalizace nákladů

- `model_reasoning_effort = "low"` pro rutinní úkoly (lint, formatter checks).
- `model_reasoning_summary = "none"` šetří output tokeny.

- `web_search = "cached"` (vs `live`).
- Profil `ci s model = "gpt-5-codex"` a `approval_policy = "never"` – minimum interakcí.
- `enable_request_compression = true` – komprese velkých requestů.
- Subagent `worker` na malé úlohy s levnějším modelem (`o4-mini`).
- `disable_response_storage = false` (default) ponecháno – prompt cache šetří input tokens.

27.4. ChatGPT subscription vs API

Aspekt	ChatGPT (Plus/Pro/Team)	API (per token)
Účtování	Měsíční flat fee	Pay-per-use
Fast mode	✓	×
Limits	Omezeny počtem zpráv / hod	Hard rate limity per minute / day
Vhodné pro	Single dev, časté použití	CI/CD, vícenásobné instance, automaty

Pro CI/CD je doporučeno API key + organization billing.

28. CI/CD a `codex-action`

28.1. GitHub Actions: `openai/codex-action`

```

name: Codex Review
on:
  pull_request:
    types: [opened, synchronize, reopened]

jobs:
  review:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      pull-requests: write
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run Codex
        uses: openai/codex-action@v1
        with:

```

```

openai-api-key: ${ secrets.OPENAI_API_KEY }
prompt-file: .github/codex/prompts/review.md
output-file: codex-output.md
safety-strategy: drop-sudo
sandbox: workspace-write
codex-args: ['--full-auto', '--cd', ".", '--profile', 'ci']

```

- name: Post comment
 - uses: marocchino/sticky-pull-request-comment@v2
 - with:
 - path: codex-output.md

28.2. Inputs **codex-action**

Input	Význam
openai-api-key	API klíč.
prompt-file	Markdown s promptem.
output-file	Kam zapsat finální zprávu.
safety-strategy	drop-sudo / keep-sudo.
sandbox	Sandbox mode.
codex-args	JSON array dalších flagů pro codex exec.

Action vystavuje output **final-message** .

28.3. Docker base image **openai/codex-universal**

Ubuntu 24.04 s předinstalovaným polyglot toolchainem (Node, Python, Go, Rust, .NET, Java, ...) a Codex CLI:

```

FROM openai/codex-universal:latest
ENV CODEX_HOME=/root/.codex
RUN --mount=type=secret,id=openai_api_key \
    OPENAI_API_KEY=$(cat /run/secrets/openai_api_key) \
    codex login --with-api-key < /dev/null

CMD ["codex", "exec", "--full-auto", "--sandbox", "danger-full-access", \
    "Run unit tests and fix failures"]

```

Build:

```
DOCKER_BUILDKIT=1 docker build --secret  
id=openai_api_key,src=$HOME/.openai_key -t my-codex .
```

28.4. GitLab CI / Jenkins

Stejný princip: instalace Codex (typicky přes `npm i -g @openai/codex` nebo `brew`) + `OPENAI_API_KEY` jako masked secret + `codex exec --full-auto`.

```
# .gitlab-ci.yml  
stages: [review]  
  
codex-review:  
  stage: review  
  image: openai/codex-universal:latest  
  variables:  
    CODEX_HOME: /root/.codex  
  script:  
    - codex exec review base-branch main --json --output-last-message  
review.md --profile ci  
  artifacts:  
    paths: [review.md]
```

28.5. Bezpečnost v CI

- `OPENAI_API_KEY` přes secret store (GH Secrets, GitLab CI variables, AWS Secrets Manager).
- Profil `ci` s `approval_policy = "never"` a `sandbox_mode = "danger-full-access"` (kontejner = izolace).
- Pinned verze action: `openai/codex-action@v1.x` (ne `@main`).
- Minimální `permissions` v GH Action (read + write jen tam, kde nutné).

28.6. Plánované (opakované) spouštění

⚠ Na rozdíl od Claude Code, Codex CLI **nemá vlastní scheduler** (nic jako `/schedule` skill). Periodické úlohy se proto staví na OS-level nástrojích: Linux **cron** (jednoduché), **systemd timer** (produkční), případně **Kubernetes CronJob** (kontejnerizované prostředí).

Vzor: `codex exec` v non-interaktivním režimu, vždy s `--profile ci` (nebo vlastním profilem s `approval_policy = "never"`) a explicitním sandboxem.

28.6.1. Linux cron

Minimální crontab:


```
# crontab -e
0 7 * * 1 cd /opt/projects/api && /usr/local/bin/codex exec --full-auto --
profile ci --output-last-message /var/log/codex/digest-$(date +%F).md
"Vygeneruj týdenní digest commitů od minulého pondělí." >>
/var/log/codex/cron.log 2>&1
```

Doporučený vzor s wrapper skriptem (lock, retry, secrets, logy):

```
#!/usr/bin/env bash
# /opt/codex-jobs/digest.sh
set -euo pipefail

# --- Konfigurace ---
PROJECT_DIR="/opt/projects/api"
LOG_DIR="/var/log/codex"
LOCK_FILE="/var/lock/codex-digest.lock"
JOB_ID="digest-$(date +%F-%H%M)"
export CODEX_HOME="/etc/codex" # sdílený config.toml, profiles

# --- Secrets ---
export OPENAI_API_KEY="$(cat /etc/codex/api-key)"
chmod 600 /etc/codex/api-key

# --- Idempotence: lock proti souběhu ---
exec 9>"$LOCK_FILE"
if ! flock -n 9; then
    echo "[$JOB_ID] Předchozí běh ještě běží, končím." >&2
    exit 0
fi

# --- Vlastní běh ---
mkdir -p "$LOG_DIR"
cd "$PROJECT_DIR"

# Retry s exponential backoffem
for attempt in 1 2 3; do
    if codex exec \
        --profile ci \
        --json \
        --output-last-message "$LOG_DIR/$JOB_ID.md" \
        "$(cat /opt/codex-jobs/prompts/digest.txt)" \
        > "$LOG_DIR/$JOB_ID.json" \
```

```

2> "$LOG_DIR/$JOB_ID.err"
then
    echo "[$JOB_ID] OK (pokus $attempt)"
    exit 0
fi
sleep $((attempt * 30))
done

echo "[$JOB_ID] Selhalo po 3 pokusech" >&2
exit 1

```

Crontab se pak smrskne na:

```
0 7 * * 1 /opt/codex-jobs/digest.sh
```

💡 Detaily, na které se pravidelně narazí:

- **PATH v cronu** je minimální. Vždy plnou cestu `/usr/local/bin/codex`, nebo na začátku skriptu `PATH=/usr/local/bin:/usr/bin:/bin`.
- **% v cronu** = newline, escapovat na `\%` nebo volat ze skriptu.
- **CODEX_HOME** musí ukazovat na místo, kde běžící uživatel může číst `config.toml` a `profiles`. Cron běží defaultně jako daný uživatel (typicky bez `$HOME`).
- **Timezone**: cron jede v systémové TZ; pro fixní TZ přidat `CRON_TZ=Europe/Prague` na začátek crontabu.
- **MAILTO**: bez nastavení posílá cron stderr emailem; `MAILTO=""` to vypne.

Doporučený profil v `~/codex/config.toml` pro cron job:

```

[profiles.ci]
approval_policy = "never"
sandbox_mode = "workspace-write"           # nebo "read-only" pro digesty
model = "gpt-5-codex"
include_plan_tool = false
disable_response_storage = true            # auditovatelnost

```

28.6.2. systemd timer

Produkční varianta. Lepší logy (`journalctl`), nativní retry, izolace pomocí systemd sandboxingu, jednotky lze stageovat / deployovat.

Service unit (`/etc/systemd/system/codex-digest.service`):

[Unit]

Description=Codex weekly commit digest

Wants=network-online.target

After=network-online.target

[Service]

Type=oneshot

User=codex

Group=codex

WorkingDirectory=/opt/projects/api

Environment=CODEX_HOME=/etc/codex

EnvironmentFile=/etc/codex/env # OPENAI_API_KEY=...

ExecStart=/usr/local/bin/codex exec \
--profile ci \
--json \
--output-last-message /var/log/codex/digest.md \
"Vygeneruj týdenní digest commitů od minulého pondělí."

StandardOutput=append:/var/log/codex/digest.out

StandardError=append:/var/log/codex/digest.err

Retry: max 3 pokusy s odstupem 60 s

Restart=on-failure

RestartSec=60

StartLimitBurst=3

StartLimitIntervalSec=600

Sandboxing systemd (nad rámec Codex Landlocku)

NoNewPrivileges=true

PrivateTmp=true

ProtectSystem=strict

ProtectHome=true

ReadWritePaths=/opt/projects/api /var/log/codex

Timer unit (/etc/systemd/system/codex-digest.timer):

[Unit]

Description=Spouští codex-digest.service pondělí v 07:00

[Timer]

OnCalendar=Mon 07:00 Europe/Prague

Persistent=true # dohnání po výpadku

```
AccuracySec=1min
RandomizedDelaySec=5min
```

[Install]

```
WantedBy=timers.target
```

Aktivace:

```
sudo systemctl daemon-reload
sudo systemctl enable --now codex-digest.timer
systemctl list-timers --all           # ověření plánu
journalctl -u codex-digest.service -f # živé logy
```

28.6.3. Kubernetes CronJob

Pro kontejnerizovaná prostředí (codex-universal image):

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: codex-weekly-digest
spec:
  schedule: "0 7 * * 1"
  timeZone: "Europe/Prague"
  concurrencyPolicy: Forbid           # ekvivalent flocku
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 7
  jobTemplate:
    spec:
      backoffLimit: 2
      activeDeadlineSeconds: 1800
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: codex
              image: openai/codex-universal:latest
              env:
                - name: OPENAI_API_KEY
                  valueFrom:
                    secretKeyRef: { name: codex-secret, key: api-key }
                - name: CODEX_HOME
```

```

    value: /etc/codex
command: ["codex", "exec", "--profile", "ci", "--json"]
args: ["Vygeneruj týdenní digest commitů od minulého pondělí."]
volumeMounts:
  - { name: config, mountPath: /etc/codex, readOnly: true }
  - { name: workspace, mountPath: /workspace }
volumes:
  - { name: config, configMap: { name: codex-config } }
  - { name: workspace, emptyDir: {} }

```

28.6.4. Secret management

Prostředí	Doporučený mechanismus
Cron na osobním stroji	~/.config/codex/env s chmod 600, sourcovat ve wrapperu.
systemd timer	EnvironmentFile=s chmod 640 root:codex.
Více serverů	HashiCorp Vault / AWS Secrets Manager + sidecar exportující env.
Kubernetes CronJob	secretKeyRef na Secret resource (raději externí, např. External Secrets Operator).
ChatGPT OAuth login	Kopírovat ~/.codex/auth.json z bezpečné enklávy; rotovat dle politiky.

 **Nikdy** nedávat `OPENAI_API_KEY` do crontabu ani do TOML profilu commitovaného do gitu.

`config.toml` je text, profily lze ale parametrizovat přes env (`api_key_env = "OPENAI_API_KEY"`).

28.6.5. Logování a rotace

- Cron: výstupy do `/var/log/codex/` , rotace přes logrotate:

```

/var/log/codex/*.log /var/log/codex/*.json /var/log/codex/*.err {
    weekly
    rotate 8
    compress
    missingok
    notifempty
    create 640 codex codex
}

```

- systemd: `journalctl -u codex-digest.service --since "1 week ago"` . Limity v `/etc/systemd/journald.conf` (`SystemMaxUse=`).
- `--json` produkuje strojově čitelný stream událostí (jq, Loki, Elastic).

- `--output-last-message FILE` zapíše poslední zprávu agenta jako markdown – ideální payload pro Slack / e-mail notifikaci.
- Sessions jsou v `~/.codex/sessions/YYYY/MM/DD/*.jsonl` ; pro audit cron jobů držte `disable_response_storage = false` u profilu.

28.6.6. Idempotence a retry

Codex `exec` může selhat z externích důvodů (rate limit, timeout MCP, síť, sandbox violation). Doporučené vzory:

- **Lock soubor** (`flock` v cronu, `concurrencyPolicy: Forbid` v K8s, `StartLimitBurst` v systemd) – brání souběhu, když interval < trvání úlohy.
- **Idempotentní prompt** – „pokud výstupní soubor pro daný den existuje, jen ho přeposli, znova negeneruj“. Hooky (§ 19) mohou tuto kontrolu vynutit shellem.
- **Retry s backoffem** – ve wrapper skriptu (`for attempt in 1 2 3`) nebo přes systemd (`Restart=on-failure` + `StartLimitBurst`).
- **activeDeadlineSeconds / --timeout** – tvrdá pojistka proti agentovi, který uvízne na MCP volání.
- **approval_policy = "never"** – jakékoli interaktivní eskalace v cron jobu znamenají hang. Musí být zakázány.
- **sandbox_mode** – pro plánované úlohy preferujte `read-only` (digest, reporting) nebo `workspace-write` s úzkým `working_directory` ; `danger-full-access` jen v ephemeral kontejneru.

28.6.7. Antipatterny

- × Cron volá `codex` bez wrapperu a logování → tichá selhání, žádný audit trail.
- × `OPENAI_API_KEY` v crontabu nebo v git-trackovaném skriptu.
- × Bez `--profile ci` – cron job dědí interaktivní defaulty (`approval_policy = "on-request"`) a hangne.
- × Bez timeoutu (`activeDeadlineSeconds` , systemd `RuntimeMaxSec`) – agent může běžet hodiny a spalovat tokeny.
- × `sandbox_mode = "danger-full-access"` mimo izolovaný kontejner.
- × Spouštění nad sdíleným repem bez locku → race condition na pracovním stromu.
- × Spoléhat na cron MAILTO jako jediný monitoring – často skončí ve spamu.

29. Best practices

29.1. Mentální model

- Začněte v `read-only` na neznámém repu. Pak povyšte na `workspace-write` až po ujištění, že agent rozumí strukturu.
- `/clear` mezi nesouvisejícími úkoly. Kontext nepomůže, jen mate.
- `/compact` zachrání tokeny v dlouhém vlákně.
- `/diff` před ukončením relace – zkontrolujte, co se reálně změnilo.

29.2. AGENTS.md hygiena

- < 200 řádků, < 64 KiB (`project_doc_max_bytes`).
- Konvence projektu, build & test příkazy, „čeho se nedotýkat“, architektonické principy.
- Aktualizujte v rámci PR, která mění setup.

29.3. Profily a providery

- `development` , `review` , `ci` , `azure` , `oss` – mít připravené.
- `--profile review` před každým code review (bezpečnější + hloubka reasoningu).
- `--profile ci` se `sandbox_mode = "danger-full-access"` jen v CI containeru.

29.4. Skills, agenty, hooks

- **Skills** pro opakované know-how (refaktor patterny, EF Core migrace, scaffolding testů).
- **Subagenty** pro úkoly s vlastním kontextem (security-reviewer, test-writer).
- **Hooks** pro deterministické vynucení (formátovač, audit, prevence secrets).

29.5. MCP servery

- Pro každý nástroj rozhodněte: **stdio** (rychlé spuštění, lokální) vs **streamable HTTP** (sdílený mezi agenty / IDE).
- Tokeny vždy z env (`bearer_token_env_var`).
- Pro CI: `--strict-mcp-config` ekvivalent v Codexu zatím neexistuje, ale lze odejmout `mcp_servers` z profilu `ci` .

29.6. Verzování konfigurace

- `.codex/config.toml` , `.codex/agents/` , `.codex/skills/` , `.codex/hooks/` , `AGENTS.md` – commitovat.
- `.codex/local/` , `.codex/secrets/` – `.gitignore` .
- `~/.codex/auth.json` , `~/.codex/memories/` – mimo repo.

29.7. Bezpečnostní rytmus

-  `OPENAI_API_KEY` jen v env, nikdy v `config.toml` .
-  `--dangerously-bypass-approvals-and-sandbox` jen v Dockeru / VM.
-  `disable_response_storage = true` pro Zero Data Retention organizace (smluvní povinnost).
-  v `[shell_environment_policy]` držte default exclude pro `*KEY*` / `*SECRET*` / `*TOKEN*` .

29.8. Workflow vzory

Vzor: „Onboarding cizího repa“

```
# Profil safe-onboarding
[profiles.safe-onboarding]
approval_policy = "untrusted"
sandbox_mode = "read-only"
```

```
model_reasoning_effort = "high"
web_search = "disabled"
```

```
codex --profile safe-onboarding "Vysvětli architekturu tohoto projektu."
```

Vzor: „Plánuj → implementuj → testuj“

```
> /permissions read-only
> Naplánuj migraci EF6 → EF Core 9.
... [plán] ...
> /permissions on-request
> Aplikuj kroky 1–3.
... [změny] ...
> Spusť `dotnet test` a oprav padající.
```

Vzor: „Multi-agent code review“

```
> /agent security-reviewer
[security-reviewer] > Audit Services/Auth.
> /agent default
> /agent test-writer
[test-writer] > Doplně pokrytí pro AuthService.
> /agent default
> /review
```

29.9. Antipatterny

- × Provoz s `danger-full-access` na hostu. Použijte Docker.
- × Hardcoded API klíč v `config.toml`.
- × AGENTS.md > 64 KiB (tichý truncation).
- × Příliš permisivní `web_search = "live"` v lokálním dev (snižuje cache hit ratio, drahé).
- × Spuštění bez profilu v CI.

30. Troubleshooting

30.1. „Sandbox denied access“

Příčina: file write mimo `cwd` + `writable_roots`, nebo `network_access = false`.

Řešení:

1. Identifikujte cestu (`~/codex/log/codex-tui-*.log`).

2. Přidejte do `[sandbox_workspace_write].writable_roots` .
3. Pro síť: `network_access = true` (nebo přepněte na `danger-full-access` v Dockeru).

30.2. „Model not found / 404“

- Zkontrolujte `model_provider` v `config.toml`.
- Pro Azure: `query_params.api-version` a `base_url` (musí obsahovat `/openai`).
- Pro OSS: běží Ollama na `http://localhost:11434/v1` ?

30.3. „AGENTS.md byl truncatován“

Symptom: instrukce na konci souboru se nedostanou do kontextu.

Řešení:

- `project_doc_max_bytes = 131072` (128 KiB).
- Rozdělit AGENTS.md po sekcích a načíst nejzávažnější přes `@reference` .

30.4. „MCP server failed to start“

```
RUST_LOG=mcp=debug codex
```

- Stdio: zkontrolujte `command` a oprávnění.
- HTTP: ověřte, že `experimental_use_rmcp_client = true` v `[features]` .
- Bearer: `bearer_token_env_var` vyžaduje, aby env var existoval v parent shellu.

30.5. „Codex se neptá, ale měl by“

- `approval_policy = "never"` + `sandbox_mode = "danger-full-access"` = Codex nikdy nečeká.
- Zkontrolujte aktivní profil (`/status`).
- Zkontrolujte CLI flagy (`--full-auto` přebíjí config).

30.6. „Stream se zasekl“

Zvýšte `stream_idle_timeout_ms` v provideru:

```
[model_providers.openai]
stream_idle_timeout_ms = 600000 # 10 minut
```

Nebo zkuste jiný provider (Azure má jinou síťovou trasu).

30.7. „Pomalý start“

- Velké AGENTS.md → ořezat.
- Mnoho registrovaných MCP serverů → `required = false` na nepovinné, `startup_timeout_sec` nižší.

- Memories se loadují → `disable_on_external_context = true` .

30.8. Updaty selhávají

```
codex doctor    # diagnostika (pokud existuje v dané verzi)
codex update    # od v0.128+
```

Pokud `codex update` chybí, použijte balíčkový manažer (`brew upgrade codex` , `npm i -g @openai/codex@latest`).

30.9. „Keymapy mi nefungují“

- `/keymap` ukáže aktuální mapování.
- V tmux / screen některé sekvence (Ctrl+R, Shift+Enter) nelze předat – povolte `xterm-keys` nebo upravte tmux config:

```
set-option -g xterm-keys on
```

30.10. „Mám dva účty, jak se přepínat?“

- `CODEX_HOME=~/.codex-personal codex login` (přihlášení do alternativního home).
- V druhém terminálu `CODEX_HOME=~/.codex-work codex` .
- Nebo `OPENAI_API_KEY=<klíč> codex` pro per-shell override.

Příloha A – Kompletní reference flagů

A.1. Globální

Flag	Hodnota	Význam
<code>-m, --model</code>	model id	Vynutit model.
<code>-p, --profile</code>	název	Aplikovat profil.
<code>-c, --config</code>	KEY=VAL	Inline override (JSON sémantika).
<code>--cd</code>	path	Pracovní adresář.
<code>-i, --image</code>	path	Příloha obrázku.
<code>--oss</code>	–	Použít OSS provider.
<code>--full-auto</code>	–	Auto-approval + workspace-write.
<code>--ask-for-approval</code>	mode	untrusted / on-request / never.
<code>--sandbox</code>	mode	read-only / workspace-write / danger-full-access.

--dangerously-bypass-approvals-and-sandbox	–	 Vypne brzdy.
--search	–	web_search = "live" pro daný běh.
--enable	feature	Zapnout feature flag.
--login	–	Vyžádat login flow.
--remote	ws://...	Remote app-server.
--help, -h	–	Nápověda.
--version, -V	–	Verze.

A.2. codex exec

Flag	Význam
--json	JSONL stream událostí.
-o, --output-last-message <FILE>	Zapsat finální zprávu.
--output-schema <FILE>	JSON Schema pro výstup.
--include-plan-tool	Plan tool.
--skip-git-repo-check	Povolit běh mimo git repo.

A.3. codex login

Flag	Význam
--with-api-key	API key ze stdin.
--device-auth	Device-code flow.

A.4. codex resume

Flag	Význam
--last	Nejnovější ze cwd.
--all	Napříč libovolným adresářem.

Příloha B – Reference klíčů `config.toml`

B.1. Root

Klíč	Typ	Účel
------	-----	------

model	string	Default model.
model_provider	string	Klíč do [model_providers].
model_reasoning_effort	enum	none/minimal/low/medium/high/xhigh.
model_reasoning_summary	enum	auto/concise/detailed/none.
model_verbosity	enum	low/medium/high.
model_supports_reasoning_summaries	bool	Provider podporuje summaries.
model_context_window	int	Override, když se nedá odvodit.
model_max_output_tokens	int	Hard cap output tokenů.
approval_policy	string/object	untrusted/on-request/never nebo { granular = {...} }.
sandbox_mode	enum	read-only/workspace-write/danger-full-access.
web_search	enum	cached/live/disabled.
notify	array	Externí notify program.
disable_response_storage	bool	True pro ZDR.
project_doc_max_bytes	int	Limit AGENTS.md.
project_doc_fallback_filenames	array	Fallback jména paměťových souborů.
preferred_auth_method	enum	apikey/chatgpt.
exec_command_timeout_ms	int	Timeout shell exec.
experimental_codex_linux_sandbox_exe	path	Override sandbox runneru.

B.2. [features]

Klíč	Typ	Účel
shell_tool	bool	Zapnutí shell toolu.
apps	bool	Apps framework.
codex_hooks	bool	Hooks.
unified_exec	bool	Nový exec runner.
shell_snapshot	bool	Snapshot shell stavu.
multi_agent	bool	Subagenti / threads.
personality	bool	/personality.

fast_mode	bool	Fast mode (ChatGPT subscr.).
enable_request_compression	bool	Komprese requestů.
skill_mcp_dependency_install	bool	Skills mohou zavádět MCP.
memories	bool	Persistentní memories.
experimental_use_rmcp_client	bool	Streamable HTTP MCP.

B.3. [sandbox_workspace_write]

Klíč	Typ	Účel
writable_roots	array	Cesty s povoleným write.
network_access	bool	Síť uvnitř sandboxu.
exclude_tmpdir_env_var	bool	Vyloučit \$TMPDIR.
exclude_slash_tmp	bool	Vyloučit /tmp.

B.4. [shell_environment_policy]

Klíč	Typ	Účel
inherit	enum	all/core/none.
ignore_default_excludes	bool	Vypnout default filtry.
exclude	array	Glob patterny k vyloučení.
include_only	array	Pouze tyto.
set	object	Hard-coded hodnoty.

B.5. [history]

Klíč	Typ	Účel
persistence	enum	save-all/none.
max_bytes	int	Cap velikosti.

B.6. [tui]

Klíč	Typ	Účel
theme	string	Téma.
notifications	bool/array	Zapnutí.

notification_method	enum	auto/osc9/bel.
notification_condition	enum	unfocused/always.
animations	bool	Animace v TUI.
show_tooltips	bool	Tooltipy.

B.7. [memories]

Klíč	Typ	Účel
generate_memories	bool	Codex generuje.
use_memories	bool	Codex používá.
disable_on_external_context	bool	Vyloučit MCP/web search thready.

B.8. [agents]

Klíč	Typ	Účel
max_threads	int	Paralelní thready.
max_depth	int	Hloubka delegace.
job_max_runtime_seconds	int	Timeout subjobu.

B.9. [mcp_servers.<name>]

Stdio:

Klíč	Typ	Účel
command	string	Spustitelný binární.
args	array	Argumenty.
env	object	Statické env.
env_vars	array	Whitelist env z parent shellu.
startup_timeout_sec	int	Timeout startu.
tool_timeout_sec	int	Timeout per-tool.
required	bool	Selhání = fatální.
experimental_environment	enum	local/remote.

HTTP:

Klíč	Typ	Účel
url	string	MCP endpoint.
bearer_token	string	Statický token.
bearer_token_env_var	string	Token z env.

B.10. [model_providers.<name>]

Klíč	Typ	Účel
name	string	Display name.
base_url	string	API endpoint.
wire_api	enum	responses/chat.
env_key	string	Název env s API klíčem.
query_params	object	Static query params.
http_headers	object	Static headers.
env_http_headers	object	Header → env var.
request_max_retries	int	Retry pro 5xx.
stream_max_retries	int	Retry pro broken stream.
stream_idle_timeout_ms	int	Idle timeout.

B.11. [profiles.<name>]

Libovolná podmnožina root klíčů.

B.12. [projects."<path>"]

Klíč	Typ	Účel
trust_level	enum	trusted/untrusted.

Příloha C – Reference environment proměnných

Proměnná	Význam
CODEX_HOME	Base dir pro config / sessions / auth. Default ~/.codex.
OPENAI_API_KEY	API key (default env_key pro openai provider).
OPENAI_ORGANIZATION	Org ID (mapováno na OpenAI-Organization header).

OPENAI_PROJECT	Project ID (OpenAI - Project header).
AZURE_OPENAI_API_KEY	Pro Azure provider.
OPENROUTER_API_KEY	Pro OpenRouter.
RUST_LOG	Log level (error/warn/info/debug/trace).
VISUAL / EDITOR	Externí editor pro Ctrl+G.
CODEX_DISABLE_AUTOUPDATE	(neověřeno) Vypne codex update.

Nedostala-li se proměnná do tabulky, ale má vliv (např. `HTTP_PROXY` , `HTTPS_PROXY` , `NO_PROXY`), Codex je respektuje skrz request stack.

Příloha D – TOML šablony

D.1. Profil

```
[profiles.review]
model = "gpt-5-pro"
model_reasoning_effort = "high"
model_reasoning_summary = "detailed"
approval_policy = "never"
sandbox_mode = "read-only"
web_search = "cached"
```

D.2. Provider

```
[model_providers.azure]
name = "Azure OpenAI"
base_url = "https://YOUR.openai.azure.com/openai"
wire_api = "responses"
query_params = { "api-version" = "2025-04-01-preview" }
env_key = "AZURE_OPENAI_API_KEY"
env_http_headers = {
  "OpenAI-Organization" = "OPENAI_ORGANIZATION",
  "OpenAI-Project" = "OPENAI_PROJECT"
}
request_max_retries = 4
stream_max_retries = 5
stream_idle_timeout_ms = 300000
```


D.3. MCP server (studio)

```
[mcp_servers.docs]
command = "node"
args = ["/mcp/docs-server.mjs"]
env = { LOG_LEVEL = "info" }
env_vars = ["DOCS_TOKEN"]
startup_timeout_sec = 10
tool_timeout_sec = 60
required = true
```

D.4. MCP server (streamable HTTP)

```
# Vyžaduje [features] experimental_use_rmcp_client = true
[mcp_servers.figma]
url = "http://127.0.0.1:3845/mcp"
bearer_token_env_var = "FIGMA_TOKEN"
```

D.5. Subagent (`.codex/agents/security-reviewer.toml`)

```
name = "security-reviewer"
description = "Provádí security audit (OWASP Top 10)."
```

`developer_instructions` = ""

Jsi expert na bezpečnost web aplikací. Analyzuj všechny vstupní body a podle OWASP Top 10 označ rizika. Nepiš změny v produkčním kódu; jen reportuj a navrhní opravy.

""

```
model = "gpt-5-pro"
sandbox_mode = "read-only"
approval_policy = "never"
model_reasoning_effort = "high"
```

D.6. Skill (`SKILL.md`)

```
---
name: ef-core-migrations
description: Generuje a kontroluje EF Core migrace v .NET 9 projektech.
---
```

Jsi expert na EF Core 9 migrace.

Postup

1. Identifikuj DbContext.
2. ``dotnet ef migrations list`` → pending změny.
3. Navrhni jméno migrace v PascalCase.
4. ``dotnet ef migrations add <Name>``.
5. Verifikuj Up/Down a označ destruktivní operace.

Reference

@references/ef-style-guide.md

D.7. Plugin manifest (`.codex-plugin/plugin.json`)

```
{
  "name": "csharp-helpers",
  "version": "0.2.0",
  "description": "Pomocníci pro C#/.NET 9 vývoj.",
  "author": "Acme",
  "license": "MIT",
  "skills": ["../skills/ef-core-migrations", "./skills/scaffold-test"],
  "mcpServers": {
    "csharp-docs": { "command": "node", "args": ["../mcp/csharp-docs.mjs"] }
  },
  "hooks": {
    "PostToolUse": [
      { "matcher": "apply_patch", "command": ["dotnet", "format"] }
    ]
  }
}
```

D.8. Starter pack pro `~/ .codex/config.toml`

```
# --- Základ ---
model = "gpt-5-codex"
model_provider = "openai"
approval_policy = "on-request"
sandbox_mode = "workspace-write"
model_reasoning_effort = "medium"
web_search = "cached"
```

```

preferred_auth_method = "apikey"
project_doc_max_bytes = 131072

# --- Sandbox ---
[sandbox_workspace_write]
network_access = false
writable_roots = []

# --- Shell env ---
[shell_environment_policy]
inherit = "core"
exclude = ["AWS_*", "AZURE_*"]
set = { CI = "0", DOTNET_NOLOGO = "1" }

# --- Historie a TUI ---
[history]
persistence = "save-all"
max_bytes = 10000000

[tui]
theme = "dark"
notifications = true
notification_method = "auto"
notification_condition = "unfocused"

# --- Features ---
[features]
memories = true
codex_hooks = true
multi_agent = true
fast_mode = false
experimental_use_rmcp_client = true
enable_request_compression = true

# --- Memories ---
[memories]
generate_memories = true
use_memories = true
disable_on_external_context = true

# --- Profily ---
[profiles.development]
model = "gpt-5.1-codex"

```

```
approval_policy = "on-request"
sandbox_mode = "workspace-write"
```

[profiles.review]

```
model = "gpt-5-pro"
model_reasoning_effort = "high"
approval_policy = "never"
sandbox_mode = "read-only"
```

[profiles.ci]

```
approval_policy = "never"
sandbox_mode = "danger-full-access"
model_reasoning_effort = "low"
```

[profiles.openrouter]

```
model = "anthropic/claude-3.5-sonnet"
model_provider = "openrouter"
```

```
# --- Provider ---
```

[model_providers.openai]

```
name = "OpenAI"
base_url = "https://api.openai.com/v1"
wire_api = "responses"
env_key = "OPENAI_API_KEY"
```

[model_providers.openrouter]

```
name = "OpenRouter"
base_url = "https://openrouter.ai/api/v1"
wire_api = "chat"
env_key = "OPENROUTER_API_KEY"
```

[model_providers.oss]

```
name = "Local Ollama"
base_url = "http://localhost:11434/v1"
wire_api = "chat"
```

```
# --- MCP ---
```

[mcp_servers.docs]

```
command = "node"
args = ["/mcp/docs-server.mjs"]
env_vars = ["DOCS_TOKEN"]
startup_timeout_sec = 10
tool_timeout_sec = 60
```

```
# --- Notifikace ---
notify = ["/bin/bash", "/Users/me/.codex/hooks/notify.sh"]

# --- Trust ---
[projects."/Users/me/work/safe-repo"]
trust_level = "trusted"
```

Příloha E – Glossář pojmů

Pojem	Význam
AGENTS.md	Markdown s persistentními instrukcemi pro Codex agenta v daném scope.
App-server	Codex jádro vystavené přes JSON-RPC 2.0 (stdio / WebSocket).
Approval policy	Pravidlo, jak často Codex žádá o souhlas (untrusted / on-request / never / granular).
Codex Cloud	Hostovaná verze Codexu s vlastními úlohami; výsledky lze stáhnout přes <code>codex apply</code> .
Codex-action	Oficiální GitHub Action pro spouštění Codexu v PR.
Codex-universal	Oficiální Docker image (Ubuntu 24.04 + polyglot toolchain + Codex).
Custom prompt	(Deprecated) markdown prompt v <code>~/ .codex/prompts/</code> , vyvolávaný <code>/prompts:<name></code> .
Device-auth	OAuth flow pro headless / SSH / Docker prostředí.
Fast mode	Rychlý mode dostupný jen s ChatGPT loginem.
Granular approval	Per-kategorie kontrola (sandbox, rules, MCP, skill, ...).
Headless	Neinteraktivní spuštění (<code>codex exec</code>).
Hooks	Shell skripty napojené na lifecycle eventy.
Landlock + seccomp	Linux sandbox primitives používané Codexem.
Marketplace	Registr pluginů (git repo s indexem).
MCP	Model Context Protocol – standard pro tooly a zdroje dat.
Memories	Persistentní strukturované poznámky generované Codexem.
Multi-agent	Více paralelních threadů agentů v rámci jedné session.
Plugin	Distribuovatelný balíček (skills + hooks + MCP + apps).
Profile	Pojmenovaná sada konfiguračních klíčů.
rmcp	Rust MCP klient (streamable HTTP).

Sandbox mode	Úroveň izolace běhu (read-only / workspace-write / danger-full-access).
Seatbelt	Apple sandbox technologie (sandbox-exec).
Skill	Instructional package s autodiskoverem; novější náhrada za custom prompts.
Subagent	Agent vyvolaný hlavním agentem v izolovaném vlákně.
Trust level	Per-project flag (trusted/untrusted) určující, zda se aplikuje project-scope config.
Wire API	Protokol komunikace s providerem (responses/chat).
Workspace-write	Sandbox mode povolující write v cwd a writable_roots.
Zero Data Retention (ZDR)	Účet bez ukládání requestů na serverech OpenAI; vyžaduje disable_response_storage = true.

Příloha F – Užitečné zdroje a odkazy

- Repozitář: <https://github.com/openai/codex>
- Konfigurace: <https://github.com/openai/codex/blob/main/docs/config.md>
- Getting started: <https://github.com/openai/codex/blob/main/docs/getting-started.md>
- Exec: <https://github.com/openai/codex/blob/main/docs/exec.md>
- Project AGENTS.md: <https://github.com/openai/codex/blob/main/AGENTS.md>
- CHANGELOG: <https://github.com/openai/codex/blob/main/CHANGELOG.md>
- Releases: <https://github.com/openai/codex/releases>
- App-server protocol: <https://github.com/openai/codex/blob/main/codex-rs/app-server/README.md>
- GitHub Action: <https://github.com/openai/codex-action>
- Docker image: <https://github.com/openai/codex-universal>
- Vývojářská centrála: <https://developers.openai.com/codex>
- Komunitní průvodci: <https://blakecroxley.com/guides/codex>, <https://shipyard.build/blog/codex-cli-cheat-sheet/>, <https://codex.danielvaughan.com/>

Konec dokumentu.